

Advanced Database Systems CoSc3011_

June 15, 2025

Compiled by: Natan Asrat

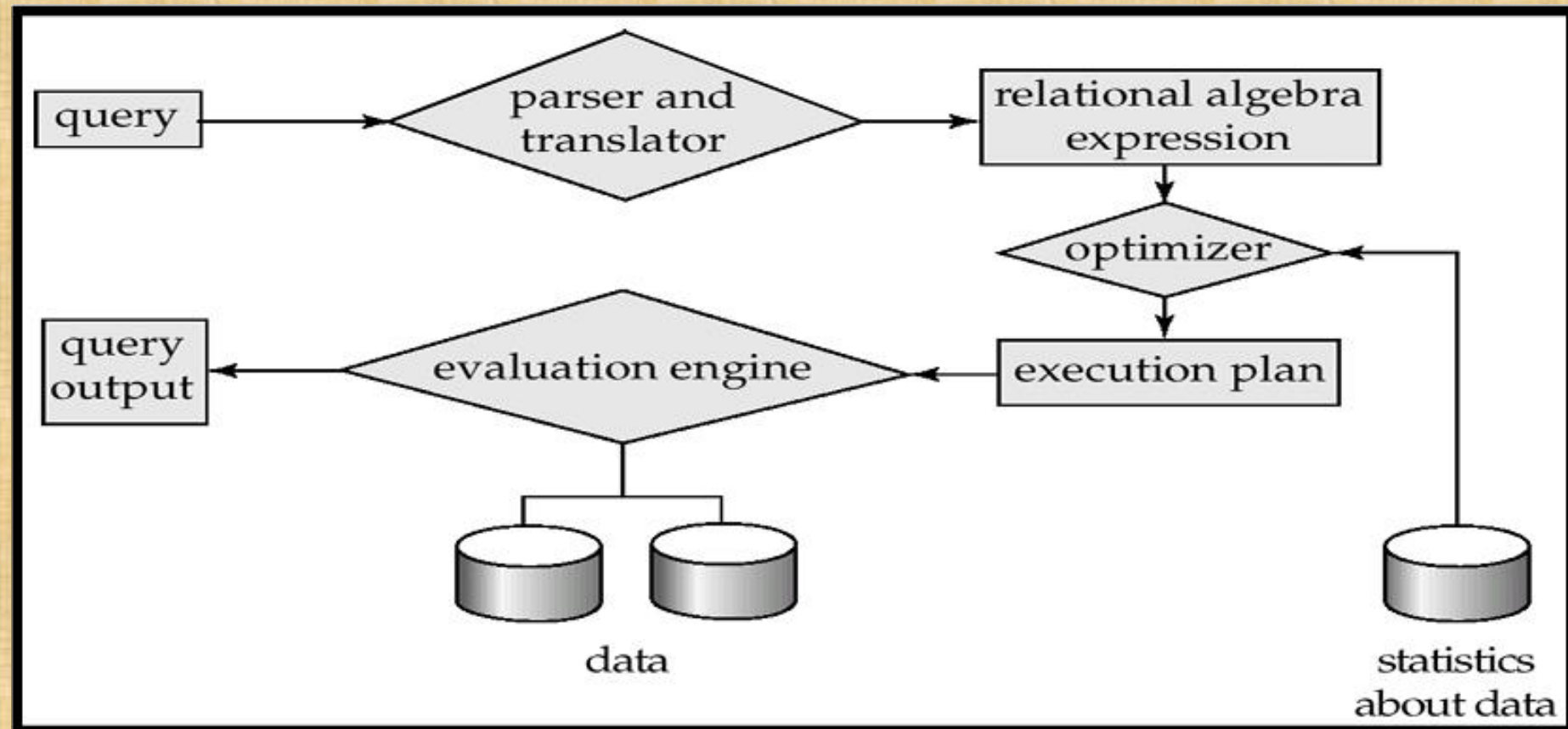
https://www.linkedin.com/in/natan_asrat

Advanced Database Systems

- Query Processing and Optimization
 - Query processing: steps required to transform high level SQL query into a correct and efficient strategy for execution and retrieval

Advanced Database Systems

- Query Processing and Optimization
 - Query Processing Steps:



Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps:
 - Decomposition: transforming a high level query into a relational algebra query; check that the query is syntactically and semantically correct
 - consists of parsing and validation

Advanced Database Systems

- Query Processing and Optimization – Query Processing Steps – Decomposition Stages
 - Analysis: lexical and syntactical analysis, based on attributes, data type
 - Query tree will be built:
 - leaf node for base relations
 - non-leaf nodes for relations produced by relational algebra operations
 - root node for the result of the query
 - Activities: analyze query with compiler techniques, verify relations and attributes exist, and verify operations

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Decomposition Stages
 - Normalization: convert the query into a normalized form. The predicate WHERE will be converted to Conjunctive (**A**) or Disjunctive (**V**) Normal form.
 - Semantic Analysis: to reject normalized queries that are not correctly formulated or contradictory. i.e. (Catalog="BS" ^ Catalog= "CS")
 - Simplification: detect redundant qualifications, eliminate common sub-expressions, and transform the query to a semantically equivalent effective computed form

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps
 - Optimization: find the sequence of steps that produces the answer efficiently
 - Optimizer: chooses an optimal plan for evaluating the request
 - Narrow down intermediate result sets quickly.
SELECT and PROJECTION before JOIN
 - Use access structures (indexes)

Advanced Database Systems

- Query Processing and Optimization – Query Processing Steps – Optimization Approaches
 - Heuristics Approach: uses relationship between the operators and characteristics of relational algebra (Properties of individual operators & Association between operators)
 - Transformation rules:
<https://www.geeksforgeeks.org/query-optimization-in-relational-algebra/>

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach - Transformation rules:

1. Cascade of Selection:

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

- applying condition $\theta_1 \wedge \theta_2$ is expensive

2. Commutativity of Selection Operations:

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

- it does not matter whether we apply σ_1 first or σ_2

3. Cascade of Projection:

$$\pi_{L_1}(\pi_{L_2} \dots (\pi_{L_n}(E)) \dots) = \pi_{L_1}(E)$$

- a series of projections is meaningless, outermost projection applies

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach - Transformation rules:

4. Commutativity of Selection:

- $\pi_{L_1}(\sigma_\theta(E)) = \sigma_\theta(\pi_{L_1}(E))$ - with projection
- $\sigma_\theta(E_1 \times E_2) = E_1 \bowtie_\theta E_2$ - with cross product
 - cross product is expensive - applying a selection operation after that scans $m * n$ entries,
 - select only those entries in the cross product which satisfy the Theta condition, without evaluating the entire cross product first

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach - Transformation rules:

5. Commutativity of Theta Join and Cross product:

- $E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$
- $R \times S = S \times R$

6. Commutativity of Selection with Theta Join

- $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = \sigma_{\theta_1}(E_1) \bowtie_{\theta} \sigma_{\theta_2}(E_2)$
 - where θ_1 contains the attributes of only E_1 and θ_2 contains attributes of only E_2 , apply selections individually
- $\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$
 - If θ_0 contains attributes from only E_1 , apply this selection to E_1

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach - Transformation rules:

7. Commutativity of Projection and Theta Join

- $\pi_{L_1} \cup_{L_2} (E_1 \bowtie_{\theta} E_2) = (\pi_{L_1}(E_1)) \bowtie_{\theta} (\pi_{L_2}(E_2))$
 - if L_1 is a projection involving columns of only E_1 , and L_2 another projection involves the columns of only E_2 , individually apply the projections on both the tables before joining
- $\pi_{L_1} \cup_{L_2} (E_1 \bowtie_{\theta} E_2) = \pi_{L_1} \cup_{L_2} ((\pi_{L_1} \cup_{L_3}(E_1)) \bowtie_{\theta} (\pi_{L_2} \cup_{L_3}(E_2)))$
 - Introduce L_3 if θ depends on a column not in L_1 and L_2

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach - Transformation rules:

8. Commutativity of Set Operations:

- $E_1 \cup E_2 = E_2 \cup E_1$
- $E_1 \cap E_2 = E_2 \cap E_1$
- $R - S \neq S - R$

9. Associativity of Theta Join, Cartesian Product, Union, and Intersection

- $(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$
- $(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$
- $(R \bowtie_{C_1} S) \bowtie_{C_2} T \equiv R \bowtie_{C_1} (S \bowtie_{C_2} T)$
- $(R \times S) \times T \equiv R \times (S \times T)$

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach - Transformation rules:

10. Commuting Selection with Set Operations: reduces the number of comparisons

- $\sigma_P(E_1 - E_2) = \sigma_P(E_1) - \sigma_P(E_2)$
- $\sigma_P(E_1 \cup E_2) = \sigma_P(E_1) \cup \sigma_P(E_2)$
- $\sigma_P(E_1 \cap E_2) = \sigma_P(E_1) \cap \sigma_P(E_2)$

11. Commuting Projection with Union: optimal

- $\pi_L(E_1 \cup E_2) = \pi_L(E_1) \cup \pi_L(E_2)$

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach:
 - Sequence for Applying Transformation rules:
 - Rule 1 - cascade selection
 - Rule 2, Rule 4, Rule 6 - commutativity of selection
 - Rule 9 - associativity
 - Rule 3 (cascade projection), Rules 4, 7, 11 - commutativity of projection
 - Using Heuristics: generate an initial internal representation (parser), optimize with heuristic rules, generate execution plan
 - main heuristic is to apply first the operations that reduce the size of intermediate results

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Optimization Approaches - Heuristics Approach:
 - Query block: The basic unit that can be translated into the algebraic operators and optimized
 - contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clause if these are part of the block
 - Nested queries within a query are identified as separate query blocks
 - Query graph: corresponds to a relational calculus expression; doesn't indicate order of operations to perform

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps:
 - Cost Estimation Approach to Query Optimization:
minimize the cost of processing a query

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Cost Estimation Approach
 - Cost function: I/O cost + CPU processing cost + communication cost + Storage cost
 - Use formulae that estimate costs for a number of options, and select one with lowest cost
 - Estimate and compare the costs of executing a query using different execution strategies and choose the strategy with the lowest cost estimate

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Cost Estimation Approach
 - Cost components: <https://www.geeksforgeeks.org/cost-based-optimization/>
 - Access Cost (from secondary storage): Searching, Reading, Writing
 - Memory Cost: buffers for execution of query
 - Storage Cost: cost of storing any intermediate files (intermediary results)
 - Computation Cost (operations): Searching, Sorting, Merging
 - Communication Cost: sending or communicating the query and its results.

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Execution:
 - Query Execution Plans consists of:
 - Relational algebra query tree,
 - Information about the access methods to be used for each relation
 - The methods to be used in computing the relational operators stored in the tree

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps - Execution:
 - Pipelining:
 - Pipelined evaluation: evaluate several operations simultaneously, passing the results of one operation on to the next
 - pass tuples directly to the join instead of storing results,
 - pass output of join directly to projection
 - Pipelines are executed in 2 ways:
 - Demand driven (lazy evaluation): system repeatedly requests next tuple from top level operation; between calls, operation has to maintain state so it knows what to return next
 - Producer-driven (eager pipelining): operators produce tuples eagerly and pass them up to their parents; buffer maintained between operators; if buffer is full child waits till it's free and then generates more tuples

Advanced Database Systems

- Query Processing and Optimization - Query Processing Steps:
 - Code Generation

Advanced Database Systems

- Database Security and Authorization
 - Database integrity: makes sure that (authorised) users use that information correctly
 - Integrity constraints
 - Domain constraints apply to data types
 - Attribute constraints apply to columns
 - Relation constraints apply to rows in a single table
 - Database constraints apply between tables
 - Intrarecord integrity (enforcing constraints on contents of fields, etc.)
 - Referential Integrity (enforcing the validity of references between records in the database)

Advanced Database Systems

- Database Security and Authorization
 - Constraints to protect from inconsistency:
 - Required data: some attributes must always contain a value
 - Attribute domain constraints: a set of values that are legal for an attribute to use
 - Entity integrity: primary key of any entity must be unique and can't be NULL

Advanced Database Systems

- Database Security and Authorization
 - Constraints to protect from inconsistency:
 - Referential integrity: foreign key value refers to existing occurrence in the parent entity
 - Inserting rules: row should not be inserted in the referencing (child) table unless there already exists a matching entry in the referenced table; sometimes NULL is used in FK
 - Deletion rules: row should not be deleted from the referenced table (parent) if there are matching rows in the referencing table (child); options: Restrict, Nullify, Cascade
 - Enterprise constraints: business rule that may affect the database and the data

Advanced Database Systems

- Database Security and Authorization
 - Concurrency control (ensure validity of database updates in a shared multiuser environment)
 - support access by multiple users to the same data, at the same time
 - handle problems (Lost updates, Inconsistent data states, Uncompleted changes to data)
 - Locking:
 - Levels: Database, Table, Block or page, Record, Field
 - Types: Shared (S locks), Exclusive (X locks)

Advanced Database Systems

- Database Security and Authorization –
Concurrency control:

- Part or all of a transaction can be undone using ROLLBACK

- Savepoints are places in a transaction that you may ROLLBACK

- SET TRANSACTION...;
- SAVEPOINT ALPHA;
- *SQL STATEMENTS...*
- IF (CONDITION) THEN ROLLBACK TO SAVEPOINT ALPHA;
- SAVEPOINT BETA;
- *SQL STATEMENTS...*
- IF ...;
- COMMIT;

Advanced Database Systems

- Database Security and Authorization:
 - Database security: makes sure that the user is authorized to access information

Advanced Database Systems

- Database Security and Authorization – Database security
 - Aspects to consider: Legal issues, Physical security, OS/Network security, Security policies and protocols, Encryption and passwords, DBMS security
 - DBMS Security Support: DBMS can provide some security (user accounts, user permissions)
 - Permissions and Privilege: SELECT privilege, INSERT privilege, UPDATE privilege, DELETE privilege
 - owner (creator) of a database has all privileges on all objects in the database & can grant them
 - owner (creator) of an object has all privileges on that object and can pass them
 - Authorization Rules: define access permissions on a table by table basis using GRANT & REVOKE SQL commands

Advanced Database Systems

- Database Security and Authorization – Database security
 - Privileges in SQL:
 - GRANT <privileges> ON <object> TO <users>
 - <privileges>: ALL, SELECT <columns>, INSERT <columns>, DELETE, UPDATE <columns>
 - <users>: list of usernames or PUBLIC
 - <object>: name of a table or view
 - GRANT <privileges> ON <object> TO <users> WITH GRANT OPTION
 - i.e. GRANT SELECT, UPDATE(Salary) ON Employee TO Finance
 - REVOKE <privileges> ON <object> FROM <users>

Advanced Database Systems

- Database Security and Authorization – Database security
 - View: A subset of the database presented to some set of users
 - CREATE VIEW name AS SELECT field1, field FROM table1, table2 WHERE <where clause>
 - Updates to the base tables change the views and vice-versa
 - For a view to be updatable: Should not use GROUP BY, HAVING, JOIN, DISTINCT, UNION

Advanced Database Systems

- Database Security and Authorization
 - Backup and Recovery:
 - Threats to Assets and Functions: Water, Fire, Power Failure, Mechanical breakdown or software failure, Accidental or deliberate destruction of hardware or software (hackers)
 - Kinds of Records:
 - Class I: VITAL (irreplaceable),
 - Class II: IMPORTANT (reproducible),
 - Class III: USEFUL (replaceable),
 - Class IV: NONESSENTIAL

Advanced Database Systems

- Transaction Processing Concepts
 - Transaction: atomic unit of work that is either completed in its entirety or not done at all

Advanced Database Systems

- Transaction Processing Concepts - Transaction
 - bounded with Begin and End statements
 - operations:
 - read_item(X) - reads a database item named X into a program variable
 - write_item(X) - writes the value of program variable X into the database item named X
 - system needs to keep track of when transaction starts, terminates, and commits or aborts

Advanced Database Systems

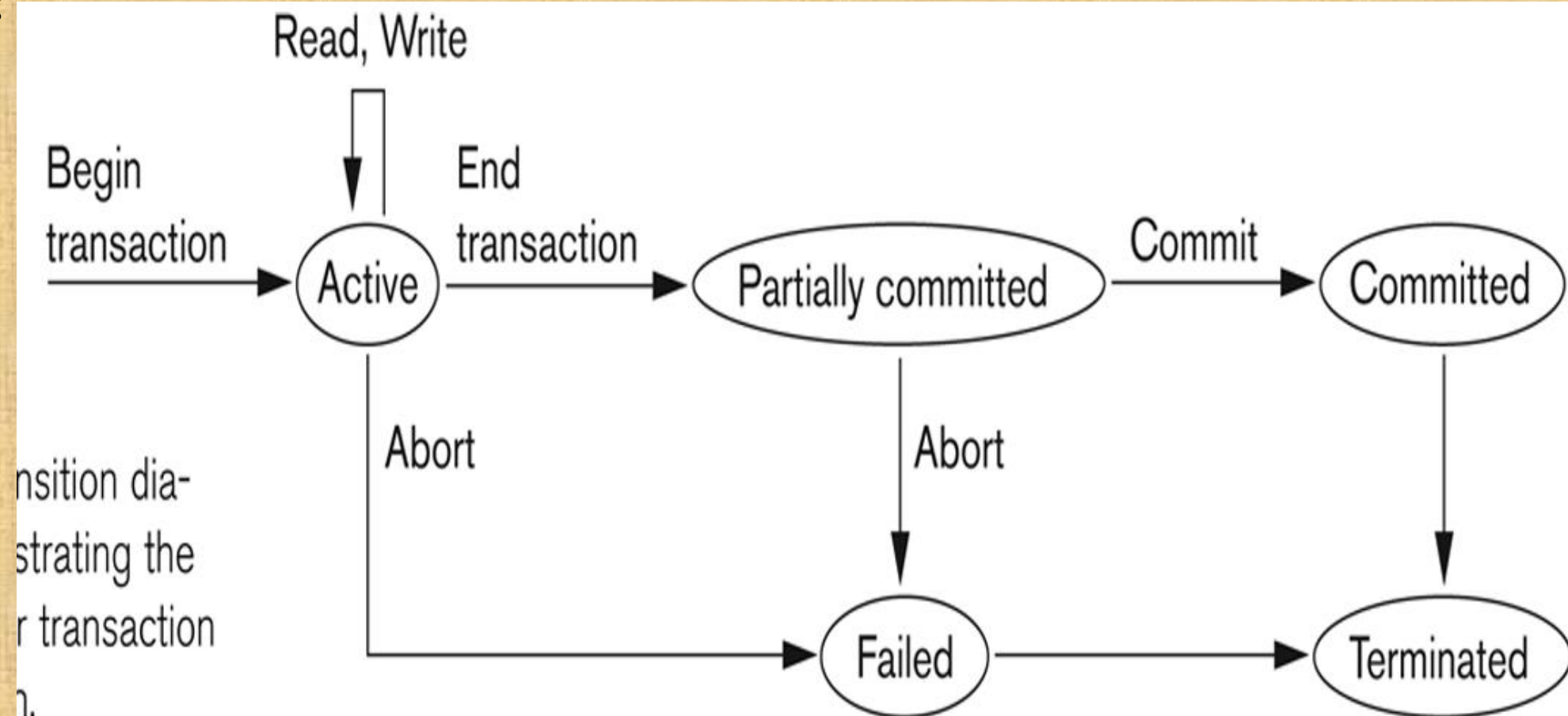
- Transaction Processing Concepts
 - Granularity of data: field, record , or a whole disk block that measure the size of the data item
 - DBMS maintains a number of buffers in the main memory holding database disk blocks which contains the database items being processed
 - When buffer is occupied and there is need for more: buffer management policy is used to choose for replacement but if the chosen buffer has been modified, it must be written back to disk before it is used

Advanced Database Systems

- Transaction Processing Concepts - Transaction states:
 - Active (beginning of a transaction execution),
 - Partially committed (end of read/write operation),
 - Committed (persistent changes),
 - Failed (aborted during active state or rechecking fails),
 - Terminated (transaction leaving the system)

Advanced Database Systems

- Transaction Processing Concepts - Transaction states:



Advanced Database Systems

- Transaction Processing Concepts:
 - Types of log record:
 - [start_transaction,T],
 - [write_item,T,X,old_value,new_value],
 - [read_item,T,X],
 - [commit,T],
 - [abort,T]

Advanced Database Systems

- Transaction Processing Concepts:
 - Properties of Transactions:
 - Atomicity (performed entirely or not),
 - Consistency preservation,
 - Isolation (updates invisible until commit),
 - Durability or permanency (no loss after commit)
 - Transaction Processing:
 - Single-User System (PC),
 - Multiuser System (multiprogramming, i.e. banks),
 - Concurrency
 - Interleaved - single CPU - round robbin,
 - Parallel - multiple CPUs)

Advanced Database Systems

- Transaction Processing Concepts – Problems of Concurrent Sharing:
 - Lost Update Problem: interleaved operations overwrite other transaction's update
 - Temporary Update/Dirty Read: transaction fails after updating item, updated item is accessed by another transaction before item is rolled back to original value
 - Incorrect Summary Problem: one transaction calculating an aggregate summary function while others are updating some of these records; aggregate could read some updates and not others

Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability:
 - Schedule/history: order of execution of operations from the various transactions
 - Sa : r2 (X) ; w2 (X) ; r1 (X) ; w1 (X) ; (schedule = transaction 2 reads X, ...)
 - Conflict in schedule: operations belonging to different transaction access the same data item X & there is at least one write_item(X)

Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Schedules classified by recoverability:
 - Recoverable: no committed transactions need to be rolled back;
 - for every pair of transactions **T_i** and **T_j**
 - **T_i** reads a data item previously written by **T_i**
 - **T_i** must commit before **T_j** commits.
 - Sc: r1(X); w1(X); r2(X); r1(Y); w2(x); c2; a1; not recoverable
 - Sd: r1(X); w1(X); r2(X); r1(Y); w2(X); w1(Y); c1; c2; recoverable
 - Se: r1(X); w1(X); r2(X); r1(Y); w2(x) ; w1(Y); a1; a2; recoverable
- Cascade-less: every transaction reads only the items that are written by committed transactions
 - Sf: r1(X); w1(X); r1(Y); c1; r2(X); w2(X); w1(Y); c2;
- Strict: transaction can neither read or write an item X until the last transaction that wrote X has committed/aborted

Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Schedules based on Serializability:
 - Serial schedule: for every transaction **T_i** participating in the schedule, all the operations of **T_i** are executed consecutively in the schedule, else non-serial
 - Serializable schedule: effect on any consistent database instance is identical to that of some complete serial schedule over the set of committed transactions in S

Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Result equivalent: two schedules produce the same final state of the database
 - Conflict equivalent: order of any two conflicting operations is the same in both schedules
 - S1: r1(x); w2(x) & S2: w2(x); r1(x) – not conflict equivalent
 - Conflict serializable: S is conflict serializable if conflict equivalent to another serializable schedule S2; implies S is serializable

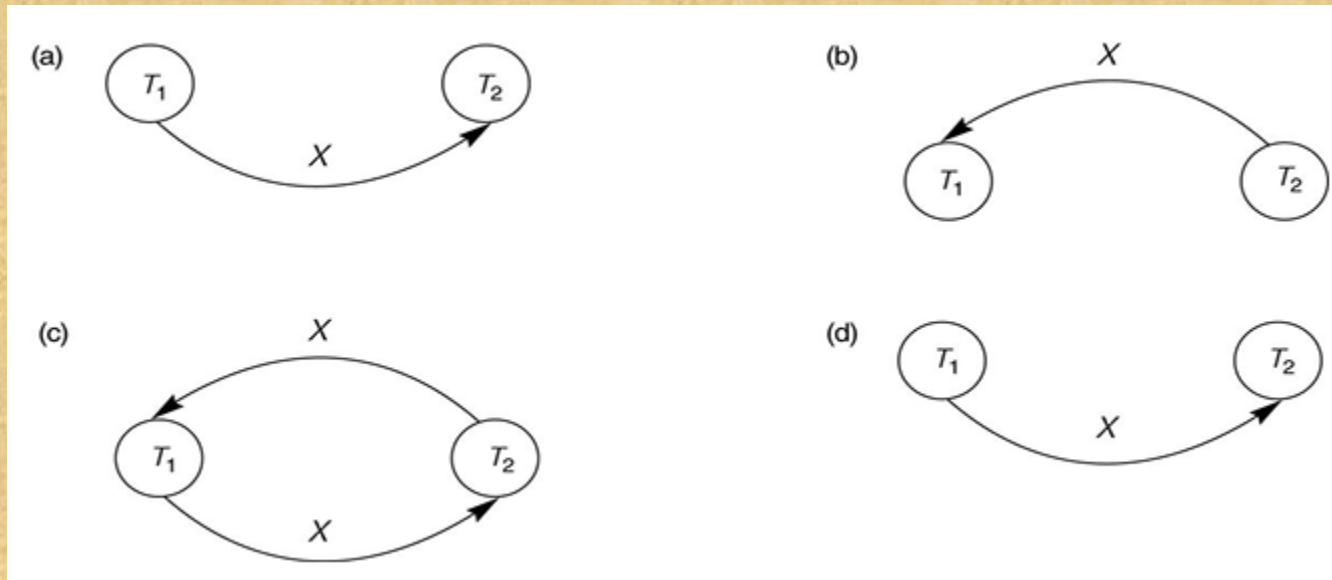
Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Result equivalent: two schedules produce the same final state of the database
 - View equivalent: same set of transactions participates in S and S' , and S and S' include the same operations of those transactions
 - If T_i reads a value A written by T_j in S_1 , it must also read the value of A written by T_j in S_2
 - for each data object A , the transaction that perform the final write on A in S_1 must also perform the final write on A in S_2

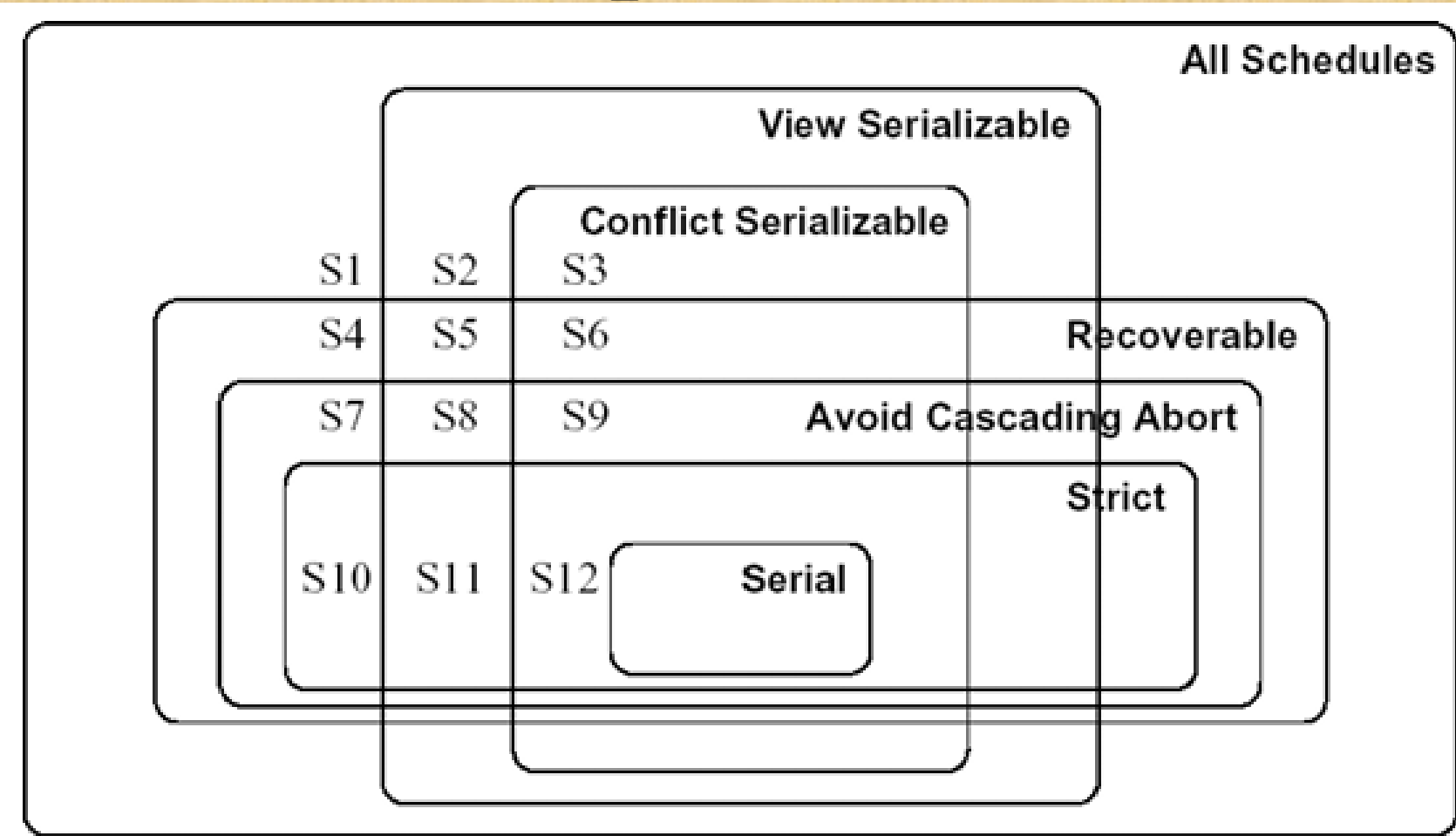
Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Result equivalent: two schedules produce the same final state of the database
 - schedule is serializable if and only if the precedence graph has no cycles

- (a) serial schedule A;
- (b) serial schedule B;
- (c) schedule C (not serializable)
- (d) schedule D (serializable, equivalent to schedule A).



Advanced Database Systems



Venn Diagram for Classes of Schedules

Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Transaction Support in SQL:
 - A single SQL statement is always considered to be atomic.

Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Transaction Support in SQL:
 - Transaction characteristics:
 - Access mode: either READ ONLY or READ WRITE (default)
 - Diagnostic size n: number of error conditions that can be held simultaneously in the diagnostic area
 - Isolation level:
 - READ UNCOMMITTED: Dirty reads allowed
 - READ COMMITTED: No dirty reads
 - REPEATABLE READ: No phantom read (multiple reads guarantee same record – no change by other transactions after reading no need to read again)
 - SERIALIZABLE (default): Fully isolated transactions

Advanced Database Systems

- Transaction Processing Concepts – Schedules and Serializability
- Transaction Support in SQL:
 - Commit Transaction SQL: COMMIT;
 - Rollback Transaction: ROLLBACK; or ROLLBACK TO SAVEPOINT my_first_savepoint;

Advanced Database Systems

- Concurrency Controlling Techniques
 - Transaction Processor is divided into:
 - Concurrency-control manager (scheduler): assure isolation of transactions
 - by maintaining locks on certain pieces of the database
 - locks prevent two transactions from accessing the same piece of data at the same time
 - Logging and recovery manager: responsible for the durability of transactions

Advanced Database Systems

- Concurrency Controlling Techniques
 - Locking: conservative (delay transactions in case they conflict with other transactions)
 - Lock: variable associated with a data item, describes it's status of possible operations
 - prevents another transaction from modifying item or even reading it in case of write lock
 - Generally, a transaction must claim a shared (read) or exclusive (write) lock on a data item before read/write
 - Unlocking: removes these permissions from the data item

Advanced Database Systems

- Concurrency Controlling Techniques – Locking
 - Shared mode: shared lock (X): More than one transaction can apply share lock on X for reading its value but no write lock can be applied on X by any other transaction.
 - Exclusive mode: Write lock (X): Only one write lock on X can exist at any time and no shared lock can be applied by any other transaction on X
 - Lock table: used by lock manager to identify locks by a transaction on a data item and pointer to the next data item locked (linked-list)

Advanced Database Systems

- Concurrency Controlling Techniques – Locking
 - Well formed transaction:
 - must lock the data item before it reads or writes to it;
 - must not lock an already locked data items and it must not try to unlock a free data item
 - Lock conversion: Lock upgrade (existing read lock to write lock), Lock downgrade (existing write lock to read lock)
 - Using such locks in transaction do not guarantee serializability of schedule on its own

Advanced Database Systems

- Concurrency Controlling Techniques - Locking
 - Two-Phase Locking Techniques: The algorithm - Transaction follows 2PL protocol if all locking operations precede first unlock operation in the transaction
 - Locking (Growing) Phase - cannot release any locks
 - Unlocking (Shrinking) Phase - cannot acquire new locks

Advanced Database Systems

- Concurrency Controlling Techniques – Locking
 - Deadlock: two or more transaction are each waiting for locks held by the other to be released

Advanced Database Systems

- Concurrency Controlling Techniques – Locking – Deadlock Solutions – Deadlock prevention protocol:
 - The conservative two-phase locking: transaction locks all data items it refers to before it begins execution
 - Limitation: restricts concurrency

Advanced Database Systems

- Concurrency Controlling Techniques - Locking - Deadlock Solutions - Deadlock prevention protocol:
 - Transaction Timestamp (TS(T)): giving each transaction a priority; lower priority transactions can't to wait for higher priority transactions (or vice versa).
 - assign priorities by a timestamp of when transaction starts up
 - oldest transaction has the highest priority

Advanced Database Systems

- Concurrency Controlling Techniques – Locking – Deadlock Solutions – Deadlock prevention protocol:
 - Transaction Timestamp:
 - If **T_i** requests a lock held by **T_j**, strategies:
 - Wait-die: older transaction is allowed to wait on a younger transaction, younger transaction requesting an item held by an older transaction is aborted
 - If **T_i** is older and **T_i** requests what **T_j** holds, it'll wait;
 - if **T_j** requests what **T_i** holds, it's aborted
 - Wound-wait: younger transaction is allowed to wait on an older one; older transaction requesting an item held by a younger transaction preempts the younger transaction by aborting it
 - if **T_i** is older and **T_i** requests what **T_j** holds, abort **T_j**
 - if **T_i** is older and **T_j** requests what **T_i** holds, it'll wait

Advanced Database Systems

- Concurrency Controlling Techniques – Locking – Deadlock Solutions:
 - Deadlock Detection and resolution: deadlocks are allowed to happen, scheduler maintains a wait-for-graph for detecting cycle, abort the transactions that have made the least work & have least number of locks and rollback
 - Timeouts: if transaction waits for a long time, system assume that may be deadlocked and aborts it; has lower overhead cost and it is simple

Advanced Database Systems

- Concurrency Controlling Techniques – Locking:
 - Starvation: a transaction consistently waits or restarted and never gets a chance
 - Solution:
 - FIFO,
 - allow transactions that waited a long time,
 - give higher priority to transactions that have been aborted many times

Advanced Database Systems

- Concurrency Controlling Techniques:
 - Timestamping - conservative:
 - Timestamp: assigned based on time in which the transactions are submitted
 - uses timestamp to serialize the execution of concurrent transactions
 - Monotonically increasing integer indicates the age of an operation or a transaction
 - larger timestamp value indicates a more recent event or operation
 - doesn't use lock, thus deadlock cannot occur

Advanced Database Systems

- Concurrency Controlling Techniques –
Timestamping:
 - Conflict check uses:
 - $\text{Read_TS}(X)$: Largest TS of transactions that read X.
 - $\text{Write_TS}(X)$: Largest TS of transactions that wrote X.

Advanced Database Systems

- Concurrency Controlling Techniques –
Timestamping – Algorithms:

1. Basic Timestamp Ordering

- Write operation $T \rightarrow \text{write_item}(X)$: abort & restart if
 - $\text{read_TS}(X) > \text{TS}(T)$ (a younger transaction read before T wrote)
 - $\text{write_TS}(X) > \text{TS}(T)$ (a younger transaction already wrote)
 - else perform write and update $\text{write_TS}(X) = \text{TS}(T)$
- Read operation $T \rightarrow \text{read_item}(X)$: abort & restart if:
 - $\text{write_TS}(X) > \text{TS}(T)$ (a younger transaction wrote before T read)
 - else perform read and set
 $\text{read_TS}(X) = \max(\text{read_TS}(X), \text{TS}(T))$

Advanced Database Systems

- Concurrency Controlling Techniques –
Timestamping – Algorithms:
 - Strict Timestamp Ordering: delaying instead of aborting.
 - Write operation $T \rightarrow \text{write_item}(X)$: If $TS(T) > \text{read_TS}(X)$:
Delay T until the reading transaction finishes.
 - Read operation $T \rightarrow \text{read_item}(X)$: If $TS(T) > \text{write_TS}(X)$:
Delay T until the writing transaction finishes.

Advanced Database Systems

- Concurrency Controlling Techniques –
Timestamping – Algorithms:
 - Thomas's Write Rule (Improved Basic TS): skip unnecessary aborts for writes
 - Write operation $T \rightarrow \text{write_item}(X)$:
 - If $\text{read_TS}(X) > \text{TS}(T)$: Abort T
 - If $\text{write_TS}(X) > \text{TS}(T)$: Ignore write, continue (newer write already exists)
 - Else: Perform write and update
 $\text{write_TS}(X) = \text{TS}(T)$

Advanced Database Systems

- Concurrency Controlling Techniques –
Timestamping:
 - Multi-version Concurrency Control: Keeps multiple versions of each data item.
 - Each version X_i has: **read_TS(X_i)** , **write_TS(X_i)**
 - Read never rejected (guaranteed version always available) .
 - Ensures **view serializability**, not conflict serializability.
 - Requires **more storage**; uses **garbage collection** to clean up

Advanced Database Systems

- Concurrency Controlling Techniques –
Timestamping - based on Timestamp Ordering:
 - Write operation $T \rightarrow \text{write_item}(X)$:
 - Let X_i be version with highest $\text{write_TS}(X_i) \leq \text{TS}(T)$
 - If $\text{read_TS}(X_i) > \text{TS}(T)$: Abort T
 - Else: Create new X_{i+1} Set $\text{read_TS}(X_{i+1}) = \text{write_TS}(X_{i+1}) = \text{TS}(T)$
 - Read operation $T \rightarrow \text{read_item}(X)$:
 - Find version X_i with highest $\text{write_TS}(X_i) \leq \text{TS}(T)$
 - Return X_i 's value
 - Set $\text{read_TS}(X_i) = \max(\text{read_TS}(X_i), \text{TS}(T))$

Advanced Database Systems

- Concurrency Controlling Techniques –
Timestamping - based on Timestamp Ordering:
 - Two-Phase Locking (Certify Lock):
 - uses **two versions**: committed X and tentative X';
 - improves concurrency,
 - may cause deadlocks,
 - prevents cascading aborts
 - Allows reading while another transaction holds **write lock**.
 - Steps:
 - X = committed version; T obtains **write lock**, creates X'.
 - Other transactions continue reading X;
 - T ready to commit → obtain **certify lock** on X'.
 - Promote X' to committed version X. Release certify lock.

Advanced Database Systems

- Concurrency Controlling Techniques:
 - Optimistic methods: assume conflict is rare and only check for conflicts at commit; If there's a conflict → **abort** and retry

Advanced Database Systems

- Concurrency Controlling Techniques - Optimistic method - Phases:
 - **Read Phase**: transaction reads committed data; writes go to **local copies**.

Advanced Database Systems

- Concurrency Controlling Techniques - Optimistic method - Phases:
 - **Validation Phase** (before commit):
 - **Did transaction conflict with others?** If yes → **Abort**; If no → Proceed to write.
 - Rules (check in order, If none match → abort **Ti**):
 - **Tj** finished writing before **Ti** started reading
 - **Ti** writes **after** **Tj** wrote AND **Ti** read & **Tj** wrote **disjoint sets**
 - **Ti** and **Tj** read/write **disjoint sets** AND **Tj** finished reading before **Ti**

Advanced Database Systems

- Concurrency Controlling Techniques - Optimistic method - Phases:
 - **Write Phase**: If validated → **Apply changes to DB**; if failed → **Restart transaction**

Advanced Database Systems

- Concurrency Controlling Techniques:
 - Multiple Granularity Locking: controls what level of data a transaction locks: DB vs. a field.
 - Granularity = size of lockable unit
 - A field of a database record
 - A database record
 - A disk block/ page
 - An entire file
 - The entire database

Advanced Database Systems

- Concurrency Controlling Techniques:
 - Multiple Granularity Locking:
 - Locking Rules (must be followed for producing serializable schedule):
 - Always lock the **root first**
 - To lock a child, the **parent must be locked**:
 - **Shared (S)/Intent Shared (IS)**
 - **Exclusive (X)/Intent Exclusive (IX)/Shared + Intent Exclusive (SIX)**
 - Once you unlock a node, **you can't lock more** → Follows 2PL (Two-Phase Locking)
 - Can unlock a node **only if none of its children are locked**

Advanced Database Systems

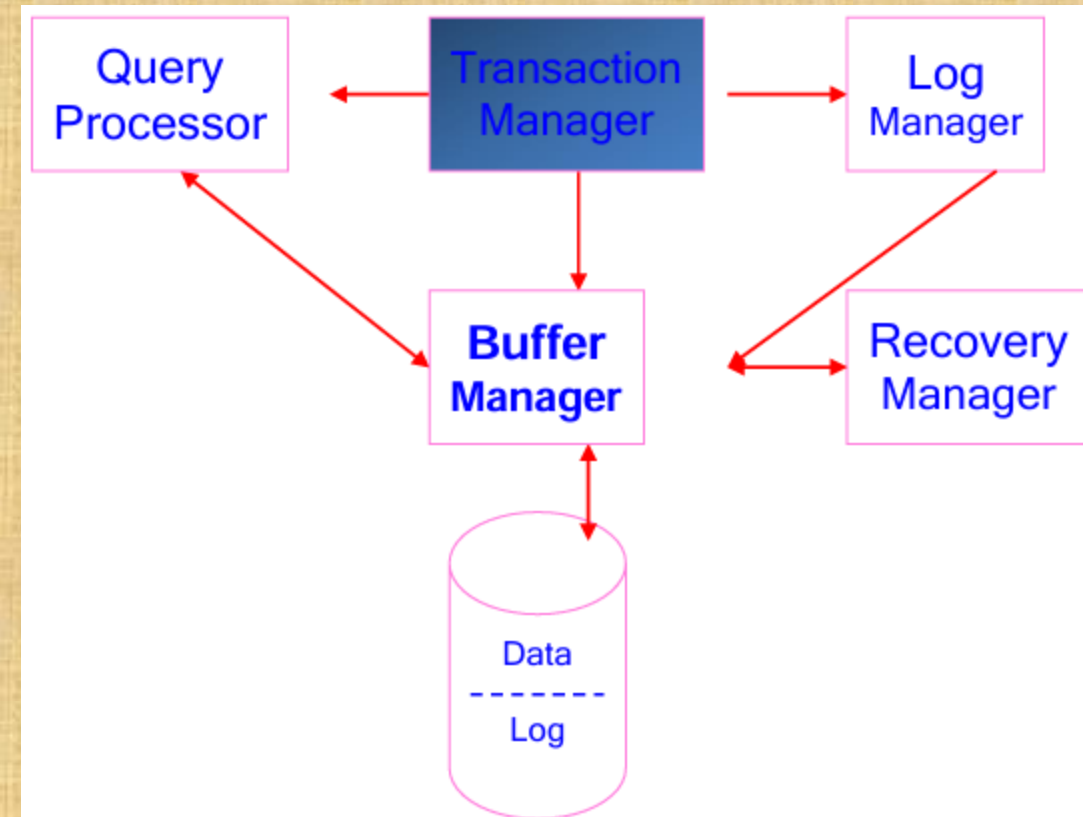
- Database Recovery Techniques
 - Purpose of Database Recovery:
 - bring the DB into the last consistent state (existed before failure)
 - preserve transaction properties (Atomicity & Durability)
 - Types of Failure:
 - Transaction failure: incorrect input, deadlock.
 - System failure: addressing error, application error, OS fault, RAM failure, etc.
 - Media failure: Disk head crash, power disruption, etc.

Advanced Database Systems

- Database Recovery Techniques
 - What causes a Transaction to fail:
 - Computer failure (system crash)
 - Transaction or system error: integer overflow, division by zero
 - Exception conditions detected by the transaction: i.e. insufficient account balance
 - Concurrency control enforcement: concurrency control aborted the transaction
 - Disk failure: disk read/write head crash, lost disk block
 - Physical problems and catastrophes

Advanced Database Systems

- Database Recovery Techniques
 - Transaction Manager: accepts transaction commands from an application
 - Transaction processor tasks: Logging (ensure durability), negotiate with buffer manager (buffer→ disk), restore the DB to consistent state (Recovery manager)



Advanced Database Systems

- Database Recovery Techniques
 - Recovery Algorithms parts: actions taken on normal execution, actions take on failure
 - Storage Structure:
 - Volatile (doesn't survive crashes - main memory, cache),
 - Nonvolatile (survives system crashes - disk, tape, flash),
Stable (approximated by maintaining multiple copies)

Advanced Database Systems

- Database Recovery Techniques
 - Stable-Storage Implementation:
 - multiple copies of each block on separate disks (can be at remote sites),
 - write info. on 1st block → write on 2nd block → successful (prevents inconsistent copies)
 - find inconsistent blocks (bad checksum)
 - Compare the two copies of every disk block - expensive
 - Record in-progress disk writes on non-volatile storage - better approach

Advanced Database Systems

- Database Recovery Techniques
 - Data Access: Physical blocks (disk), Buffer blocks (main memory)
 - Block movement: input(B) - from disk to memory, output(B) - from memory to disk
 - Each transaction **T_i** has its private work-area (local copies of all its data items)
 - **T_i**'s local copy of a data item X is called **x_i**
 - read(X) assigns the value of data item X to the local variable **x_i**.
 - write(X) assigns the value of local variable **x_i** to data item {X} in the buffer block.
 - Output(**B_x**) need not immediately follow write(X). system performs output when deems fit

Advanced Database Systems

- Database Recovery Techniques
 - Recovery and Atomicity: first output modifications to stable storage without modifying DB

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - Log based recovery
 - System log: to recover from system failure, system keeps info. on changes to system logs;
 - Strategies: Recovery from catastrophic, Recovery from non-catastrophic failure
 - if transaction hadn't committed at failure, recovery manager has to rollback
 - Transaction Log: data values prior to modification and the new value after modification are required
 - **BFIM** - BeFore Image & **AFIM** - AFter Image are stored in a sequential file (appended file) called Transaction log

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - Log based recovery
 - Data Caching: data items to be modified are first stored into database cache by the Cache Manager (CM) and after modification they are flushed (written) to the disk
 - DBMS request for read /write operation: check cache first, else get from disk

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - Log based recovery
 - Write-Ahead Logging:
 - log is necessary for in-place update (immediate or deferred)
 - For Undo: Before AFIM is flushed to DB disk, its BFIM must be written to the log and the log saved on a stable store (log disk).
 - For Redo: Before a transaction commits, all its AFIMs must be written to the log

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - Log based recovery
 - Write-Ahead Logging:
 - Combinations:
 - Steal/No-Force (Undo/Redo)
 - Steal/Force (Undo/No-redo)
 - No-Steal/No-Force (No-undo/Redo)
 - No-Steal/Force (No-undo/No-redo)

Advanced Database Systems

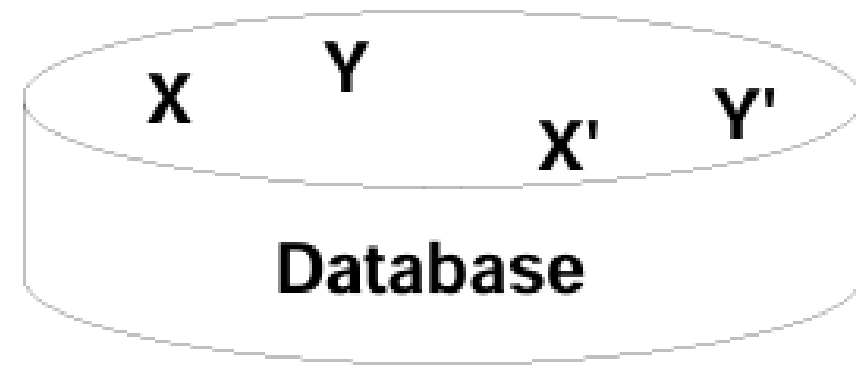
- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - Log based recovery
 - Check pointing: sync point between DB & log; reduces recovery time by only requiring redo/undo of transactions after the checkpoint
 - on failure: redo all committed transactions after checkpoint; undo all active transactions at time of crash

Advanced Database Systems

- Database Recovery Techniques – Recovery and Atomicity – Approaches:
 - Log based recovery
 - Data Update and Recovery Scheme:
 - **Deferred Update (No Undo/Redo)**: Changes are logged and only written at commit, requiring only redo during recovery.
 - **Deferred Update – Single User**: No concurrency issues, changes logged and flushed at commit.
 - **Deferred Update – Concurrent Users**: Uses active and commit tables for recovery, requiring concurrency control and redo of committed transactions only.
 - **Immediate Update – Undo/No-redo**: flush AFIMs under WAL before it commits; recovery manager undoes all transactions during recovery; no transaction is redone; even finished transaction is undone

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - Shadowing: two page tables during life of a transaction: current page and shadow page
 - pages are the same when transaction starts; update current page; shadow page isn't changed and is used for recovery
 - current page table records all updates to database; when transaction completes, current page table becomes shadow page table



X and Y: Shadow copies of data items

X' and Y': Current copies of data items

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - ARIES Algorithm - recovery information:
 - **Log Records:**
 - LSN (Log Sequence Number) - unique per log, increases monotonically, indicates the disk address of the log record
 - each data page stores the LSN of the latest log record applied
 - log written on updating a page (page's LSN set to log's LSN), commit, abort, rollback
 - transaction ID, type, page ID, BFIM...

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - ARIES Algorithm - recovery information:
 - **Transaction Table:** active transactions (ID, status - aborted, committed...) and most recent log record LSN for the transaction.
 - **Dirty Page Table:** dirty pages - page ID and LSN of earliest update to that page

Advanced Database Systems

- Database Recovery Techniques - Recovery and Atomicity - Approaches:
 - ARIES Algorithm:
 - checkpointing: ARIES uses fuzzy check pointing
 - writes a begin checkpoint record in the log (mark the start of checkpointing)
 - system gathers the contents of the Transaction Table and Dirty Page Table.
 - writes an end checkpoint record to the log, appending the captured tables.
 - write LSN of begin_checkpoint to a special file on stable storage (used to locate the latest checkpoint)

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Distributed database (DDB):
 - collection of multiple logically related database distributed over a computer network
 - DDB management system (software): manages a DDB; makes the distribution transparent
 - physical placement of data (files, relations, etc.) is not known to the user (transparency).
 - processes unit of execution (a transaction) in a distributed manner

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Distributed database (DDB) – Advantages:
 - Distribution & Network Transparency: users access data without knowing its location.
 - Replication Transparency: same data can be stored at multiple sites for faster access.
 - Fragmentation Transparency: data can be split into fragments for efficient storage and access.
 - Increased Reliability & Availability: failure of one node doesn't crash the system.
 - Improved Performance: data is kept closer to users, reducing access time.
 - Scalability: easy to add new nodes without major reconfiguration.

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Distributed database (DDB) – Disadvantages:
 - Complexity: difficult to manage due to replication, failures, and networks.
 - Higher Cost: more hardware and skilled staff.
 - Heterogeneous System Issues: machines need coordination.
 - Security & Integrity Risks: harder to control data when it's spread across locations.

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Distributed database (DDB) – Types:
 - Homogeneous: identical setup across sites, same DBMS, little or no local autonomy, can have different OS across sites
 - Heterogeneous: different vendors, managed through a single conceptual schema; degree of local autonomy is minimum – each site follows centralized access policy

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Distributed database (DDB) – Query Processing:
 - Issues: Cost of transferring data over the network, Reduce total data transfer size during query execution

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Concurrency Control and Recovery:
 - **Multiple Data Copies:** maintain consistent copies during operations and recovery.
 - **Site Failures:** system stays available even if some sites fail; recovery mechanisms must handle site restoration.
 - **Communication Failures:** failure may create network partition which would affect database availability when sites are still running
 - **Distributed Commit:** transaction may be fragmented and they may be executed by a number of sites; requires two-phase or three-phase commit protocols.
 - **Distributed Deadlock:** deadlocks can span multiple sites; resolution must be distributed.

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Distributed Concurrency Control Approaches:
 - **Primary Site Technique:** one site coordinates transactions; simple but creates a single point of failure and bottleneck; can use a backup site for fault tolerance.
 - **Primary Copy Technique:** each data item has a primary copy at a site; distributes load but requires complex directory management.
 - **Voting-Based Protocol:** no fixed coordinator; lock granted if a majority of replicas agree includes timeouts to prevent indefinite blocking

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Recovery from Coordinator Failure:
 - **With Backup Site:** Switch to backup, resume processing.
 - **Without Backup:** Abort all active transactions, elect new coordinator.

Advanced Database Systems

- Distributed Databases and Client-Server Architectures
 - Three-Tier Client/Server Architecture:
 - Layers:
 - Client (parsing and breaking down queries),
 - Web Server (enforces business rules, manages security),
 - Database Server (executes subqueries, returns data)